baboon-acoustics

Formant, F0, glottal-source and vocal-tract-length analysis of baboon vocalizations

User documentation (v1.0)

Frédéric Berthommier
CNRS, Grenoble INP, GIPSA-Lab, Université Grenoble Alpes
frederic.berthommier@gipsa-lab.grenoble-inp.fr

October 2026

Abstract

This document is the user guide of the `baboon_acoustics` Python package, the supplementary code of the article *On the Reliability of Wahoo Formant Measurements: A Response to Boë et al. (2026)* (International Journal of Primatology, 2026). It introduces the underlying acoustic concepts in plain language, walks through the installation, follows a complete tutorial on the provided recordings, documents every public function of the package, and answers frequently asked questions. No prior signal-processing knowledge is assumed.

Contents

1 Introduction **1**
1.1 What this package does . . . 1
1.2 Two ways of measuring formants . . . 1

2 Installation **2**

3 Tutorial **3**
3.1 Step 1 — load a recording . . . 3
3.2 Step 2 — track formants with Praat . . . 3
3.3 Step 3 — estimate the vocal tract length . . . 3
3.4 Step 4 — estimate F0 . . . 3
3.5 Step 5 — sliding IAIF analysis . . . 4
3.6 Step 6 — glottal source analysis . . . 4
3.7 Step 7 — F0 contour and the article's exclusion criteria . . . 4

4 API reference **5**
4.1 `baboon_acoustics.audio_io` — loading audio . . . 5
4.2 `baboon_acoustics.formants_praat` — Praat/Burg formants . . . 5
4.3 `baboon_acoustics.formants_iaif` — IAIF . . . 5
4.4 `baboon_acoustics.f0` — fundamental frequency . . . 5
4.5 `baboon_acoustics.vtl` — vocal tract length . . . 6
4.6 `baboon_acoustics.glottal` — glottal source . . . 6
4.7 `baboon_acoustics.plotting` — figures . . . 6

5 Reproducing the article figures **6**

6 FAQ and common pitfalls **7**

7 References **7**

1 Introduction

1.1 What this package does

When a mammal vocalizes, the lungs drive air through the larynx, where the vocal folds chop the flow into pulses (the *glottal source*), and the resulting sound is filtered by the cavities of the throat, mouth and nose (the *vocal tract*). This is the classic *source–filter theory* of vocal production. The package analyses recordings along these two layers:

- **Formants** are the resonant frequencies of the vocal tract. They appear as peaks in the spectral envelope and are conventionally numbered F1, F2, F3, ... from the lowest upwards. Their spacing depends on the length and shape of the tract, so they carry information about the caller's anatomy (and, when articulation occurs, about vowel-like gestures).
- **The fundamental frequency (F0)** is the repetition rate of the glottal pulses; it is the acoustic correlate of pitch.
- **The vocal tract length (VTL)** can be estimated from the formant pattern: for a tube closed at one end, the resonances fall near $F_i = (2i-1)\,c/(4\,\mathrm{VTL})$ with $i = 1, 2, 3, \ldots$ and c the speed of sound (Reby & McComb 2003).
- **The glottal source waveform** itself can be recovered by *inverse filtering*: removing the vocal-tract resonances from the recorded signal leaves (an estimate of) the glottal flow. Its shape reveals the phonation type and, in the case studied in the article, nonlinear phenomena such as subharmonics and deterministic chaos.

1.2 Two ways of measuring formants

Praat's Burg method (module `formants_praat`). Linear predictive coding (LPC) fits an all-pole filter to a short analysis window; the poles are reported as formant candidates. The implementation is Praat's *Burg* algorithm, accessed through the Parselmouth interface (Jadoul et al. 2018). The main parameter is the number of poles N: too few, and neighbouring formants merge; too many, and extra poles lock onto harmonics of the source rather than true resonances. For a tract of about 17 cm analysed up to 5.5 kHz, classical theory predicts roughly 5–6 poles; the article shows on the baboon wahoo that $N = 4$ (the setting of the original study) only tracks the central part of the call, while $N = 6$ is needed for its initial and final parts (Figure 1).

IAIF inverse filtering (module `formants_iaif`). Plain LPC assumes a spectrally flat source, which the glottal source is not. *Iterative adaptive inverse filtering* (IAIF; Alku 1991) alternates between estimating the glottal contribution (low-order LPC) and the vocal tract (high-order LPC), each iteration refining the other. The result is both a cleaner vocal-tract envelope — whose peaks are the formants — and the glottal waveform itself. IAIF needs longer windows (about 90 ms) and more computation, but it exposes source dynamics that LPC hides; on the wahoo it reveals strong departures from a regular periodic source.

2 Installation

`baboon_acoustics` requires **Python 3.9 or newer** and four packages:

```
git clone https://github.com/FBerthommier/baboon-acoustics.git
cd baboon-acoustics
python -m pip install -r requirements.txt
```

- `numpy`, `scipy` — numerical work and filtering;
- `matplotlib` — figures;

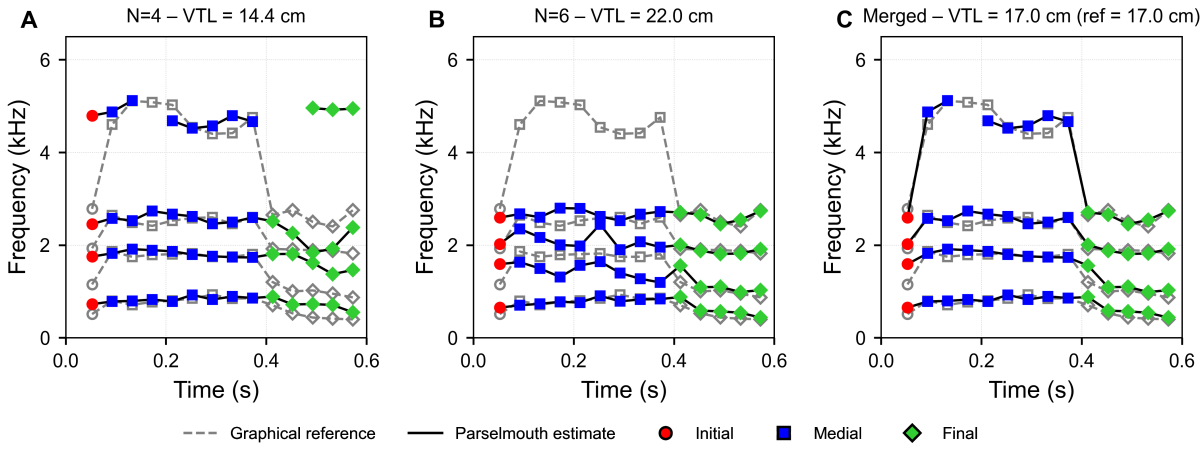


Figure 1: Formant re-analysis of the wahoo (output of `scripts/make_figure1.py`). Panels (A) and (B): Parselmouth tracks with $N = 4$ and $N = 6$ poles (grey dashed lines: reference data digitized from Boë et al. 2026). Panel (C): merged trajectory using the optimal N per segment. The N=4 setting only suits the central segment; each panel title gives the corresponding VTL estimate.

- `praat-parselmouth` — embeds the Praat sound engine; you do *not* need to install Praat separately.

Verify the installation with the test suite (about 30 seconds):

```
python -m unittest discover -s tests
```

The package is used in place: scripts and examples add the repository root to `sys.path` automatically, so simply run them from the repository folder.

3 Tutorial

This tutorial reproduces the workflow of the article on the provided recordings: `data/wahoo_4.wav` (a 0.67 s chacma baboon *wahoo* call, 44.1 kHz) and `data/The_Baboon_Bark.wav` (16 s of barks, stereo). Everything shown here is assembled in `examples/example_analysis.py`, which can be run as-is.

3.1 Step 1 — load a recording

```
from baboon_acoustics import load_audio

samples, fs = load_audio("data/wahoo_4.wav")
```

`load_audio` returns a mono, floating-point waveform and its sampling frequency. Multi-channel files are reduced to the first channel; recordings at other rates are resampled to 44.1 kHz (the default `target_fs`); with `normalize=True` the waveform is scaled to a peak amplitude of 1. Pass `normalize=False` when absolute amplitudes matter (glottal-wave displays).

3.2 Step 2 — track formants with Praat

```
from baboon_acoustics import extract_praat_formants

times, formants = extract_praat_formants("data/wahoo_4.wav",
                                         n_formants=4)
```

```
F1_hz = formants[0] # first-formant track
F1_khz = F1_hz / 1000.0 # kHz, as plotted in the article
```

The result is an array of shape `(n_formants, n_frames)` in Hz; `NaN` marks frames where Praat found no value. Frames are 40 ms apart by default (`time_step`); the analysis ceiling (`formant_ceiling`, default 5500 Hz) and the pre-emphasis frequency (50 Hz) follow the settings of the article.

3.3 Step 3 — estimate the vocal tract length

```
from baboon_acoustics import reby_mccomb_vtl

vtl_cm = reby_mccomb_vtl(formants)
print(f"VTL = {vtl_cm:.1f} cm")
```

Every valid measurement of every formant contributes one point (order, frequency), orders being the odd integers $1, 3, 5, 7, \ldots$ for F1, F2, F3, F4, … A least-squares line through the origin provides the slope, and $\mathrm{VTL} = c/(4b)$ with $c = 350$ m/s. On `wahoo_4.wav` the merged trajectory of the article yields about 17.0 cm — versus 14.4 cm (N=4) and 22.0 cm (N=6) taken separately, a first indication that formant tracking, not anatomy, drives the spread.

3.4 Step 4 — estimate F0

```
from baboon_acoustics import estimate_f0_praat

times, f0, strength = estimate_f0_praat("data/wahoo_4.wav")
```

This runs Praat's autocorrelation pitch analysis. With the default `use_prefilter=True`, the signal first passes through a Gaussian low-pass filter that emulates the *filtered autocorrelation* of recent Praat versions; on calls with strong subharmonics (the "wa" part of the wahoo), the plain autocorrelation of older versions often locks onto F0/2. Comparing both modes is a quick diagnostic for such nonlinearities:

```
_, f0_raw, _ = estimate_f0_praat("data/wahoo_4.wav",
                                 use_prefilter=False)
```

A second, source-based estimator, `estimate_f0_from_gn()`, measures F0 from the autocorrelation of a glottal waveform (step 6).

3.5 Step 5 — sliding IAIF analysis

```
from baboon_acoustics import iaif_sliding_analysis

result = iaif_sliding_analysis(samples, fs, n_formants=4)
result.times # frame times (s)
result.formants # (n_formants, n_frames) in Hz
result.f0_track # F0 per frame (Hz, NaN if unvoiced)
result.glottal_db # spectrogram of the IAIF glottal source (dB)
result.envelope_db # IAIF vocal-tract envelope |Hvt| (dB)
```

The signal is sliced into 4096-sample Hann windows (93 ms, 75 overlap) and each window is decomposed by IAIF. Figure 2 shows the complete six-panel output this analysis feeds: spectrograms with N=4 and N=6 Praat overlays (A, B), the F1–F2 trajectory against the Maximal Acoustic Space (MAS) of the species (C, D), the IAIF source with its F0 track (E), and the vocal-tract envelope with the IAIF formants (F).

3.6 Step 6 — glottal source analysis

```
from baboon_acoustics import (load_audio,
                              extract_glottal_pulse,
                              extract_glottal_train)

x, fs = load_audio("data/wahoo_4.wav", normalize=False)

t_pulse, pulse = extract_glottal_pulse(x, fs, 0.26, 0.34)
t_train, train = extract_glottal_train(x, fs)
```

`extract_glottal_pulse` isolates a single glottal cycle from a stable segment (here 260–340 ms, the vowel-like middle of the call); `extract_glottal_train` reconstructs the full pulse train by overlap-add of windowed IAIF estimates (Figure 3, top). Phase portraits of the train (`plot_phase_portrait`) expose amplitude and period irregularities: closed loops indicate periodic phonation, spread or folded trajectories indicate subharmonics or chaos (Figure 3, bottom left).

3.7 Step 7 — F0 contour and the article's exclusion criteria

Finally, `scripts/make_figure_f0.py` writes the F0 contour of the filtered autocorrelation to a CSV table and a plot (Figure 4). On the wahoo, the contour exceeds 300 Hz throughout, and F1 falls below 1.5× F0 over most of the "hoo" segment — the two exclusion criteria discussed in the article.

4 API reference

All frequencies are in Hz, times in seconds, amplitudes arbitrary unless stated. Every function is documented in its docstring; `help(module)` or `help(function)` shows the same text inside Python.

4.1 `baboon_acoustics.audio_io` — loading audio

`load_audio(path, target_fs=44100, normalize=True)`
Reads a WAV file, keeps the first channel of multi-channel recordings, converts to float64, optionally peak-normalizes to $[-1, 1]$, and resamples to `target_fs` with a polyphase filter when needed. Returns `(samples, fs)`.

4.2 `baboon_acoustics.formants_praat` — Praat/Burg formants

`extract_praat_formants(audio_path, n_formants, fs_target=44100, formant_ceiling=5500, window=0.040, time_step=0.040, pre_emphasis=50)`
Runs Praat's "To Formant (burg)". Returns `(times, formants)` with `formants` of shape `(n_formants, n_frames)` in Hz (`NaN` where undefined).

`interpolate_formants(source_times, formants, target_times)`
Linearly resamples tracks onto a new time grid, ignoring NaN frames and extending NaN outside the valid range.

`fill_leading_nan(track)`
Replaces a leading NaN (Praat's half-filled first window) by the first valid value.

4.3 `baboon_acoustics.formants_iaif` — IAIF

`lpc_levinson(x, order)`
LPC coefficients of a frame via FFT autocorrelation and the Levinson–Durbin recursion. Returns `(coefficients, energy)`.

`lpc_spectrum_db(lpc_coeffs, error_variance, n_fft=4096, fs=44100)`
Magnitude response of the LPC all-pole filter, in dB.

`iaif_frame(frame, fs, vt_order=60)`
One IAIF decomposition: returns the vocal-tract LPC filter, its prediction error, and the glottal-flow derivative estimate.

`get_frame_formants(frame, fs, vt_order=60, n_formants=4, fmax=3500, min_prominence=2.5, min_dist_hz=150)`
Formants of one frame as the `n_formants` lowest of the `n_formants + 2` most prominent peaks of the IAIF envelope below `fmax`; pads with NaN.

`iaif_sliding_analysis(x, fs, n_formants=4, ...)`
Sliding-window IAIF over the whole signal; returns an `IaifResult` named tuple (`times`, `freqs`, `spectrogram_db`, `glottal_db`, `envelope_db`, `f0_track`, `formants`).

4.4 `baboon_acoustics.f0` — fundamental frequency

`estimate_f0_from_gn(glottal, fs, f0_min=50, f0_max=500, ac_threshold=0.3)`
F0 from the autocorrelation of a glottal waveform; returns NaN for unvoiced frames.

`gaussian_lowpass_filter(samples, fs, cutoff=800, attenuation=0.03)`
Frequency-domain Gaussian pre-filter used to emulate Praat's filtered autocorrelation.

`estimate_f0_praat(audio_path, use_prefilter=True, time_step=0.01, pitch_floor=30, pitch_ceiling=800, ...)`
Praat autocorrelation pitch analysis; returns `(times, f0, strength)` for the voiced frames only.

4.5 `baboon_acoustics.vtl` — vocal tract length

`reby_mccomb_vtl(formants, c=350.0)`
Single VTL estimate (cm) from formant tracks in Hz — a `(n_formants, n_frames)` array or a sequence of tracks; NaNs ignored, any number of formants accepted. Returns NaN below two valid values.

4.6 `baboon_acoustics.glottal` — glottal source

`extract_glottal_pulse(x, fs, t_start, t_end, vt_order=60, f0_min=50, f0_max=500)`
One glottal cycle from a quasi-stationary segment (Hamming window, autocorrelation period, cut around the largest excursion). Returns `(t, pulse)` or `(None, None)` if the segment is shorter than 1024 samples.

`extract_glottal_train(x, fs, vt_order=60, win_len=4096, hop=1024)`
Full glottal train by overlap-add of windowed IAIF estimates, trimmed at the edges.

`normalize_amplitude(signal, fs, cutoff=50, order=4, eps=1e-6)`
Removes floating DC and flattens the slow envelope (Butterworth low-pass of the signal and of its magnitude); trims 512 samples at each end to discard filter transients.

4.7 `baboon_acoustics.plotting` — figures

`set_journal_style(base_size=8, tick_size=7, legend_size=6, linewidth=0.8, titlesize=None, embed_fonts=False)`
Compact Arial style of the article figures.

`add_panel_label(ax, label, ...)`
Bold panel letter, e.g. "(A)".

`setup_frequency_axis(ax)`
Time–frequency axes (s vs. kHz, 0–3.5 kHz) shared by the spectrogram panels.

`plot_f1f2_trajectory(ax, f1_khz, f2_khz, markers, colors, mas_f1_khz, mas_f2_khz)`
F1–F2 trajectory over the Maximal Acoustic Space contour (F2 axis reversed, vowel-chart convention).

`plot_phase_portrait(ax, signal, fs, ...)`
Hilbert analytic-signal portrait (optionally amplitude-normalized).

`plot_formant_comparison_panel(ax, times, formants, reference, title, letter, vtl_value, vtl_reference=None)`
One panel of Figure 1: estimated tracks and markers over a grey dashed reference.

5 Reproducing the article figures

From the repository root:

```
python scripts/make_figure1.py # Figure1.png
python scripts/make_figure_multipanel.py # 6-panel figure + glottal pulse
python scripts/make_figure_glottal.py # glottal train, "hoo" segment
python scripts/make_figure_glottal.py --segment wa # ... "wa" segment
python scripts/make_figure_f0.py # F0 table (CSV) + plot
```

Outputs land in `figures/`. On the provided data the numerical outputs are exact (the F0 table matches the archived reference to a relative difference below 10^{-9}) and the figures are visually identical to those of the article.

6 FAQ and common pitfalls

How many formants (N) should I request? A rule of thumb from quarter-wave theory: $N \approx 2{\cdot}\mathrm{VTL}{\cdot}\mathrm{F}_{\max}/c{+}1$. For a baboon tract of 17 cm and $F_{max} = 5.5$ kHz this gives $N \approx 5.5$, i.e. try $N = 5$ or 6. Extract with two or three neighbouring values and compare (`extract_praat_formants` is fast): if a formant track jumps by hundreds of Hz between settings, it is unstable — as happens with $N = 4$ on the wahoo. If your tracks look like a comb of evenly spaced peaks, N is too high and the extra poles follow harmonics.

Formants below 1.5×F0 are unreliable. Praat (and LPC in general) cannot separate a formant from a strong nearby harmonic. Track F0 as well (`estimate_f0_praat`) and discard frames where $F_1 < 1.5\,F_0$ — one of the exclusion criteria applied in the article.

Which sampling rate? Recordings are resampled to 44.1 kHz by default because the IAIF parameters (window of 4096 samples, vocal-tract order 60) are expressed at that rate. For recordings of signals far below 20 kHz you may pass `target_fs=16000` to `load_audio` — the Praat branch then needs matching `formant_ceiling` (e.g. 5000 for baboons, well below the Nyquist of any audio recording).

The file is long — IAIF is slow. The sliding analysis costs roughly 0.3 s per 0.1 s of audio at 44.1 kHz on a typical laptop. Slice the waveform first (`x = x[int(1.2 * fs):int(2.0 * fs)]`) and analyse the segment of interest; Praat-based functions, by contrast, handle minutes of audio in seconds (the 16 s bark file takes a few seconds).

F0 looks locked at half (or twice) the expected value. This is the classic subharmonic (or octave) error of autocorrelation pitch trackers on calls with nonlinear dynamics. Compare `estimate_f0_praat(..., use_prefilter=True)` and `use_prefilter=False`; systematic discrepancies indicate subharmonics in the source — inspect the glottal train (`extract_glottal_train`) and its phase portrait.

I get NaN formants / NaN VTL. NaNs are normal where the analysis cannot find a value (low signal energy, unvoiced frames). `reby_mccomb_vtl` ignores them, and returns NaN only when fewer than two valid measurements remain in the whole input — check your segment bounds if that happens.

Can I analyse species other than baboons? Yes. Only the MAS contour and the segment boundaries in the figure scripts are baboon-specific; every analysis function is generic. Adjust `fmax` / `formant_ceiling` and the F0 search range to your species.

7 References

- Berthommier, F. (2026). On the Reliability of Wahoo Formant Measurements: A Response to Boë et al. (2026). *International Journal of Primatology* (accepted 1 October 2026).
- Alku, P. (1991). Glottal wave analysis with pitch synchronous iterative adaptive inverse filtering. *Proceedings of Eurospeech 1991*, 1081–1084.
- Boë, L.-J., Laporte, M., Sawallis, T. R., Badin, P., & Schwartz, J.-L. (2026). Stalking the wild formant: How to reliably measure formants in primate vocalizations. *International Journal of Primatology*, 47(3), 799–841. https://doi.org/10.1007/s10764-026-00551-4
- Fitch, W. T., Anikin, A., Pisanski, K., Valente, D., & Reby, D. (2025). Formant analysis of vertebrate vocalizations: Achievements, pitfalls, and promises. *BMC Biology*, 23, Article 92. https://doi.org/10.1186/s12915-025-02188-w
- Jadoul, Y., Thompson, B., & de Boer, B. (2018). Introducing Parselmouth: A Python interface to Praat. *Journal of Phonetics*, 71, 1–15.
- Reby, D., & McComb, K. (2003). Anatomical constraints generate honesty: acoustic cues to age and weight in the roars of red deer. *Animal Behaviour*, 65(3), 519–530.

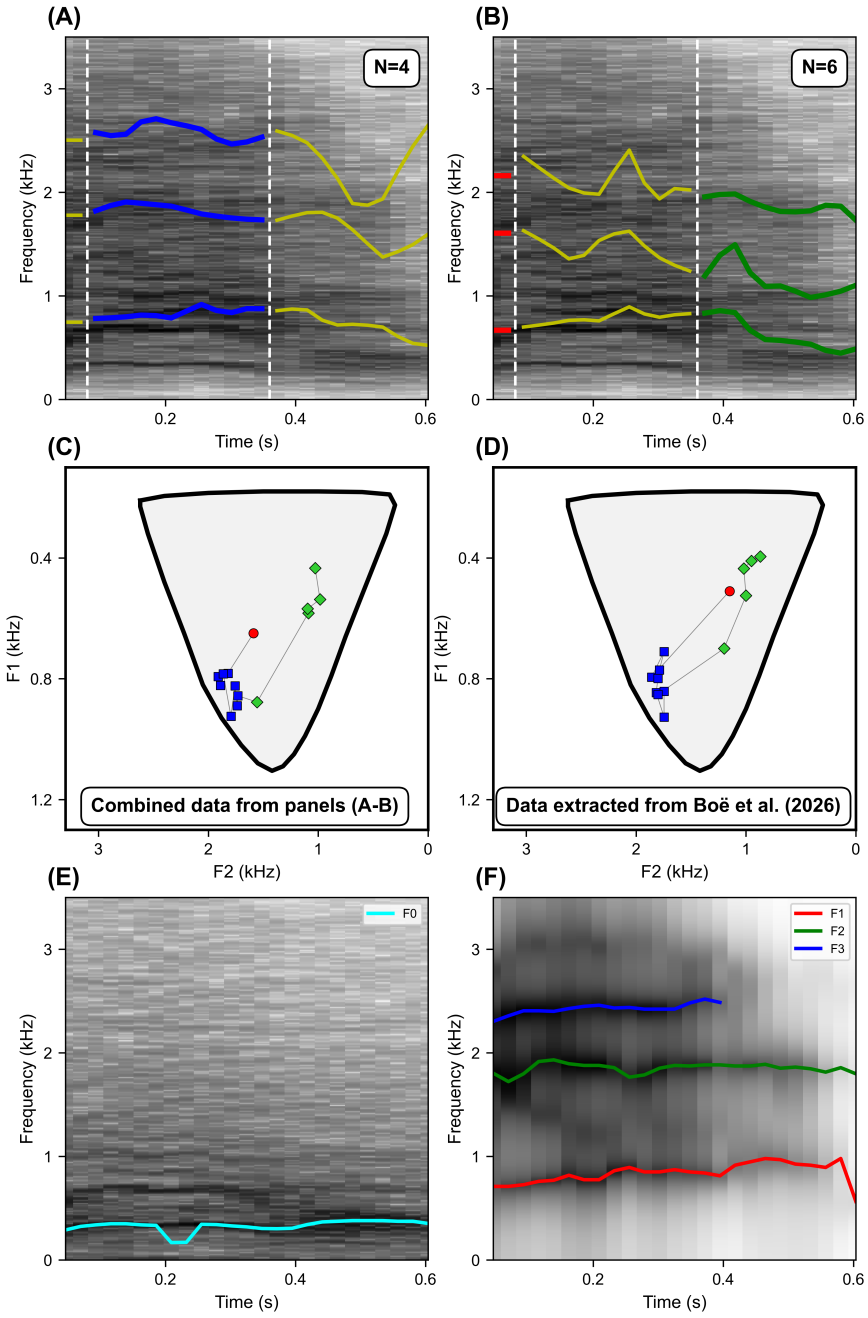


Figure 2: Six-panel analysis of `wahoo_4.wav` (output of `scripts/make_figure_multipanel.py`). The F1–F2 trajectories of panels (C)–(D) are drawn over the Maximal Acoustic Space contour of the chacma baboon; markers encode the position within the call (circle: initial frame; squares: medial; diamonds: final).

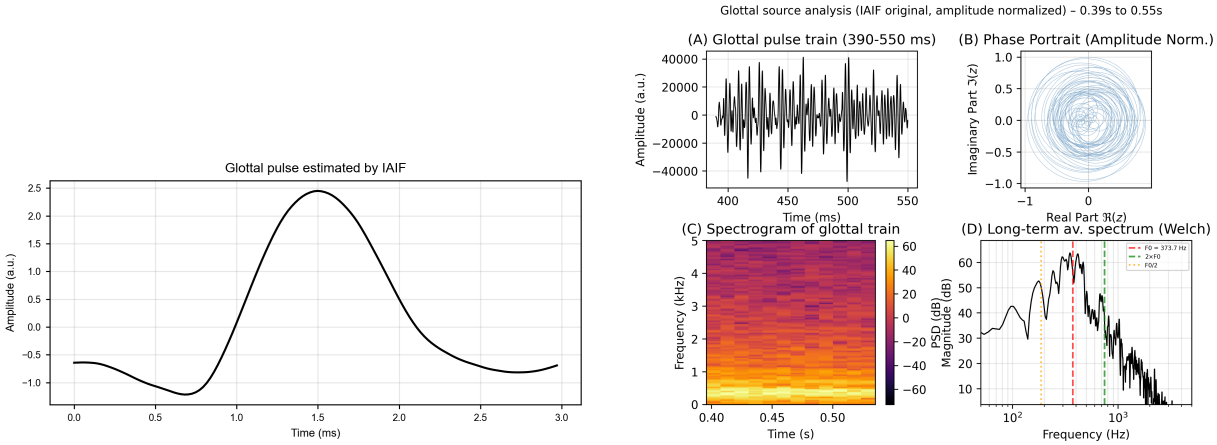


Figure 3: Left: single glottal pulse estimated by IAIF. Right: four-panel glottal-source analysis of the "hoo" segment (output of `scripts/make_figure_glottal.py`): waveform, Hilbert phase portrait, spectrogram, and long-term average spectrum with F0 markers.

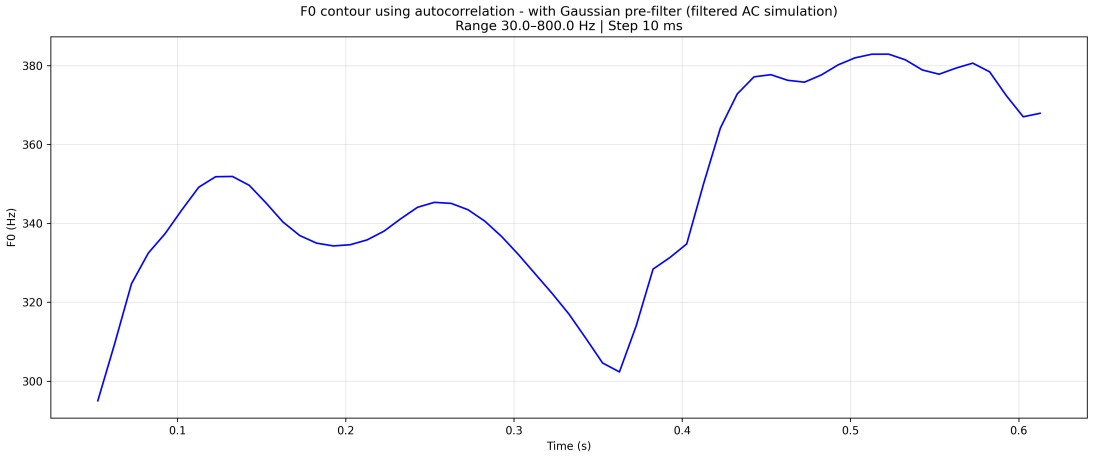


Figure 4: F0 contour of `wahoo_4.wav` obtained with the Gaussian pre-filtered autocorrelation (output of `scripts/make_figure_f0.py`).